WEEK 4 · LESSON 19 OF 24 · HANDOUT

AI Agents: LLMs That Use Tools and Take Steps

Study time: **about 60 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=2j26a5dmCnI — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyOhio 2025: Demystifying AI Agents with Python Code — https://www.youtube.com/watch?v=jOEdb9HnwsQ

🎓 **Go deeper:** PyData Virginia 2025: Mastering LLMs — from Prompt Engineering to Agentic AI — https://www.youtube.com/watch?v=jmwLzX_ltbQ

Your goal in this lesson: Explain what makes a system an agent, build an agent loop with tools, step limits, error handling and human approval for risky actions, and decide when a simple workflow is better.

Workflow or agent?

In Week 2 the model called one tool and we answered. An **agent** goes further: the model decides **which step to take next**, looks at the result, and repeats until the task is done.

	Workflow	Agent
Who decides the steps?	You, in code	The model, at run time
Predictable?	Very	Less
Cost and speed	Low, fixed	Higher, varies
Good for	Known processes (classify → retrieve → answer)	Open tasks where the steps depend on what is found

Start with a workflow. Use an agent only when the steps really cannot be known in advance. Many "agent" products are mostly well-designed workflows.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

```
import re

def search_handbook(query, k=2):
    words = set(re.findall(r"[a-z0-9]+", query.lower())) - {"the", "a", "is", "how", "what",
"do", "i", "for", "of", "to"}
    scored = sorted(chunks, key=lambda c: -len(words & set(re.findall(r"[a-z0-9]+",
c["text"].lower()))))
    return [c["text"] for c in scored[:k]]

print(search_handbook("meeting room extra hours price")[0][:80], "...")
```

The tools

Each tool is a normal Python function plus a description the model reads. Tools that **change** something (booking, paying, sending) are marked as risky:

```
import ast
import json
import operator

def calculator(expression):
    ops = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul,
           ast.Div: operator.truediv, ast.USub: operator.neg}
    def ev(node):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in ops:
            return ops[type(node.op)](ev(node.left), ev(node.right))
        if isinstance(node, ast.UnaryOp) and type(node.op) in ops:
            return ops[type(node.op)](ev(node.operand))
        raise ValueError("only numbers and + - * / are allowed")
    return str(ev(ast.parse(expression, mode="eval").body))

BOOKINGS = []
def book_meeting_room(day, hours):
    BOOKINGS.append({"day": day, "hours": hours})
    return f"Booked a meeting room on {day} for {hours} hour(s)."

TOOLS = [
    {"name": "search_handbook", "description": "Search the Kigali Tech Hub handbook for rules,
prices and opening hours.",
     "input_schema": {"type": "object", "properties": {"query": {"type": "string"}},
"required": ["query"]}},
    {"name": "calculator", "description": "Do arithmetic with + - * /, for example '3 *
5000'.",
     "input_schema": {"type": "object", "properties": {"expression": {"type": "string"}},
"required": ["expression"]}},
    {"name": "book_meeting_room", "description": "Book a hub meeting room. This makes a real
booking.",
     "input_schema": {"type": "object", "properties": {"day": {"type": "string"}, "hours":
{"type": "integer"}},
                      "required": ["day", "hours"]}},
]
FUNCTIONS = {"search_handbook": lambda query: "\n".join(search_handbook(query)),
             "calculator": calculator, "book_meeting_room": book_meeting_room}
RISKY = {"book_meeting_room"}

print(calculator("3 * 5000"), "|", calculator("(10000 + 25000) / 2"))
```

Notice that `calculator` does **not** use Python's `eval()`. Text coming from a model must be treated like text from a stranger: never run it as code.

The agent loop

```
AUTO_APPROVE = False          # in Colab you can replace approve() with input("Allow? y/n ") ==
"y"

def approve(name, args):
    print(f"   APPROVAL NEEDED: {name}({args}) -> {'approved' if AUTO_APPROVE else 'refused'}")
    return AUTO_APPROVE

def run_agent(task, max_steps=6):
    messages = [{"role": "user", "content": task}]
    for step in range(1, max_steps + 1):
        response = client.messages.create(model=MODEL, max_tokens=800, tools=TOOLS,
messages=messages,
                                          system="You help members of the Kigali Tech Hub. Use
the tools. "
                                                 "Look up prices in the handbook; never guess
them.")
        messages.append({"role": "assistant", "content": response.content})
        if response.stop_reason != "tool_use":
            return "".join(b.text for b in response.content if b.type == "text")
        results = []
        for block in response.content:
            if block.type != "tool_use":
                continue
            print(f"step {step}: {block.name}({json.dumps(block.input)})")
            if block.name in RISKY and not approve(block.name, block.input):
                output, error = "The user did not approve this action.", True
            else:
                try:
                    output, error = FUNCTIONS[block.name](**block.input), False
                except Exception as exc:                      # tell the model, do not crash
                    output, error = f"Tool error: {exc}", True
            results.append({"type": "tool_result", "tool_use_id": block.id, "content": output,
"is_error": error})
        messages.append({"role": "user", "content": results})
    return "Stopped: the step limit was reached before the task was finished."

print(run_agent("How much do 3 extra meeting room hours cost? Search the handbook, then use the
calculator."))
```

The loop is the whole idea of an agent: **call the model → run the tools it asks for → send back the results → repeat**. Four safety features are built in:

1. **A step limit** — a confused agent cannot loop forever (or spend forever).
2. **Errors go back to the model** — it can correct itself instead of crashing your app.
3. **Human approval** for actions that change the world.
4. **A log** of every step, so you can see why it did what it did.

A risky action needs approval

```
print(run_agent("Please book a meeting room for Friday for 2 hours."))
print("bookings made:", BOOKINGS)
```

With `AUTO_APPROVE = False` the booking is refused and the list stays empty. In a real product the approval would be a button the member presses: "Book Friday, 2 hours? Yes / No".

Designing good agents

- **Few, clear tools** with precise descriptions and input schemas. Ten vague tools confuse the model.
- **Least privilege:** a tool should only be able to do what the task needs (read-only search cannot delete anything).
- **Return useful errors** ("day must be Monday–Saturday") so the model can retry correctly.
- **Test with an evaluation set** like Lesson 17, including tasks the agent should refuse.
- **Standard connections:** the Model Context Protocol (MCP) is an open standard for connecting models to tools and data sources, so one tool server can be reused by many apps.

Key terms

- **Agent** — a system where the model chooses the next step in a loop with tools.
- **Workflow** — a fixed sequence of steps decided in code.
- **Agent loop** — model call → tool calls → results → repeat.
- **Step limit** — a maximum number of loop turns.
- **Human-in-the-loop approval** — a person confirms risky actions.
- **Least privilege** — giving tools only the power they need.

Remember this: An agent is a loop where the model picks tools until the task is done; keep it safe with few clear tools, step limits, errors fed back, logs and human approval for anything that changes the world.

Name: _____ Date: _____

Activity

Task 1

Add a tool `opening_hours(day)` and ask "Can I work at the hub on Sunday?".

Task 2

Set `max_steps=1` and run the first task. What happens?

Task 3

Give `book_meeting_room` a check that refuses Sundays with a helpful error. Watch the model react.

Task 4

Write the same "price of 3 extra hours" task as a fixed workflow without an agent. Which is simpler?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What makes a system an agent rather than a workflow?
   - A. It runs on a GPU
   - B. It has a chat window
   - C. The model decides the next step in a loop, using tools
   - D. It uses Python
2. What should you usually try first?
   - A. No testing
   - B. A complex agent
   - C. A simple workflow
   - D. Fine-tuning
3. Why does the calculator tool not use Python's eval()?
   - A. eval is slow
   - B. Text from a model must never be run as code
   - C. eval cannot add
   - D. It is not installed
4. Why give an agent a step limit?
   - A. To make it smarter
   - B. To save memory
   - C. The API requires it
   - D. So a confused agent cannot loop and spend forever
5. Which tool should need human approval?
   - A. Adding two numbers
   - B. Reading opening hours
   - C. Searching the handbook
   - D. Booking a meeting room
6. What should a tool do when the input is wrong?
   - A. Return a helpful error the model can use to retry
   - B. Return nothing
   - C. Delete data
   - D. Crash the app

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.